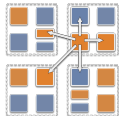


Benchmarking and developing NAMD With Charm++ and Reconverse

Ritvik Rao



Why Charm++ is moving to Reconverse

- Converse is the messaging and scheduling layer for Charm++
 - 30+ years old
 - Need for maintainable software stack
 - Old, unused code
 - Difficult to identify/fix bugs (even with Claude)
 - Lack of new features for multithreading in process



Reconverse replaces only Converse

Charm++ on Converse

NAMD

Charm++ runtime

Converse: scheduler, messaging

Machine layers: UCX, MPI, OFI, ...

replaced
by



Charm++ on Reconverse

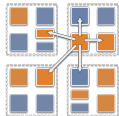
NAMD (unchanged)

Charm++ runtime (unchanged)

Reconverse: scheduler, messaging

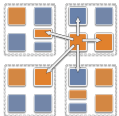
LCI

LCW (over MPI)



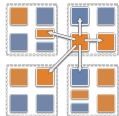
What Reconverse adds over Converse

- Separated from Charm++ into a separate repo
- Supports new backends such as LCI
 - Enables new features such as lci_ndevices, variable progress polling, etc.
- Removes need for comm thread
- New support for GPU communication, including collectives and one-sided communication
- New registered scheduler (only register messaging queues used by NAMD)



Small changes to the NAMD code base

- Minor changes to NAMD build to add Reconverse options
- One additional line to register a trace function in NAMD startup
- But otherwise, NAMD code runs as-is



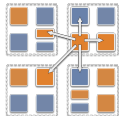
Building and running NAMD

- Build Charm++ for a Reconverse target
 - `./build charm++ reconverse-linux-arm8 --with-production`
- Configure NAMD with a Reconverse arch file
 - `./config Linux-ARM64-g++-reconverse --charm-arch reconverse-linux-arm8 --with-cuda`
- Launch with `srun` or `mpirun`, one process per GPU
 - `namd3 +ppn 32 +lci_ndevices 8 +setcpuaaffinity in.namd`
 - No `+commmap`: no core is set aside for a comm thread
- Across nodes, set `LCI_ATTR_PACKET_SIZE=32768`



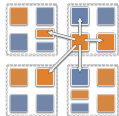
NAMD feature status on Reconverse

- Runs today on Vista (NVIDIA GH200)
 - GPU-offload, 1 to 16 nodes
 - GPU-resident, single node
 - memopt with parallel input: STMV 8M and 64M
 - Replica exchange (+replicas)
 - Load balancing: centralized and hybrid
- Not yet tested
 - Colvars and other global-force methods
 - Alchemical FEP/TI, trajectory output at scale

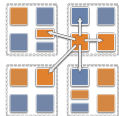
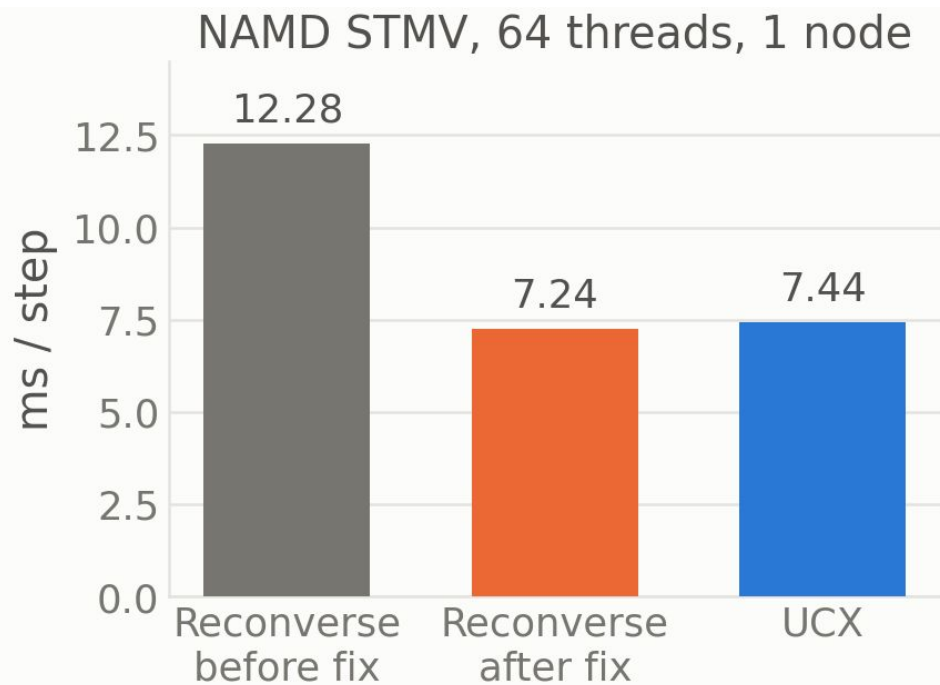
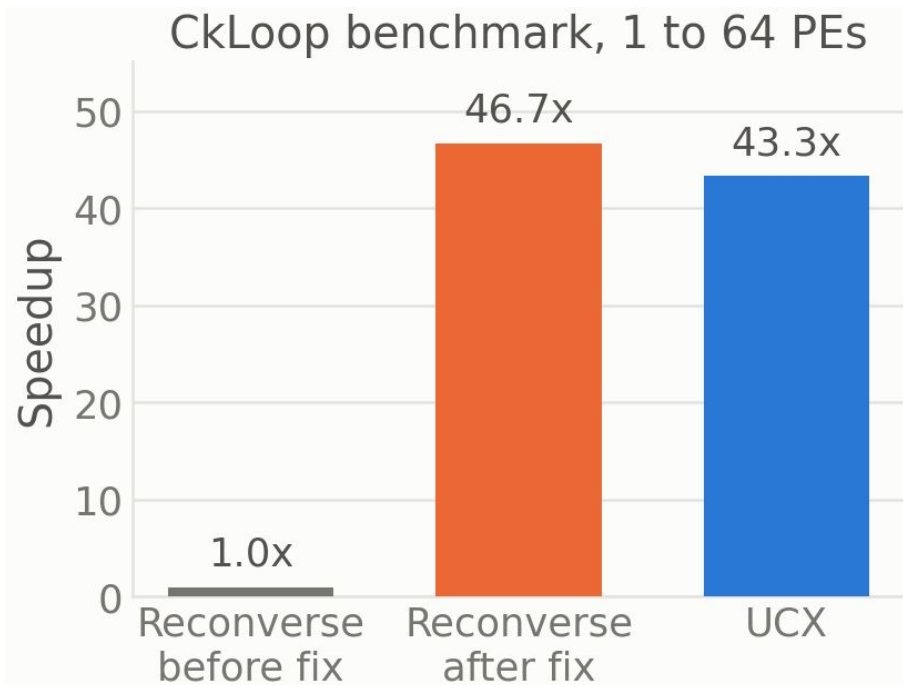


Configurations run and remaining

Machine	Configuration	Nodes	Status
Vista (NVIDIA GH200)	GPU-resident	1	Done
Vista (NVIDIA GH200)	GPU-offload	1-16	Done
Vista (NVIDIA GH200)	memopt, STMV 8M	2	Done
Vista (NVIDIA GH200)	Hybrid load balancer	2	Done
Vista (NVIDIA GH200)	Replica exchange	1-2	Done
Vista (NVIDIA GH200)	GPU-offload	32+	To do
Frontier (AMD MI250X)	HIP GPU build		To do
Aurora (Intel GPU Max)	SYCL GPU build		To do



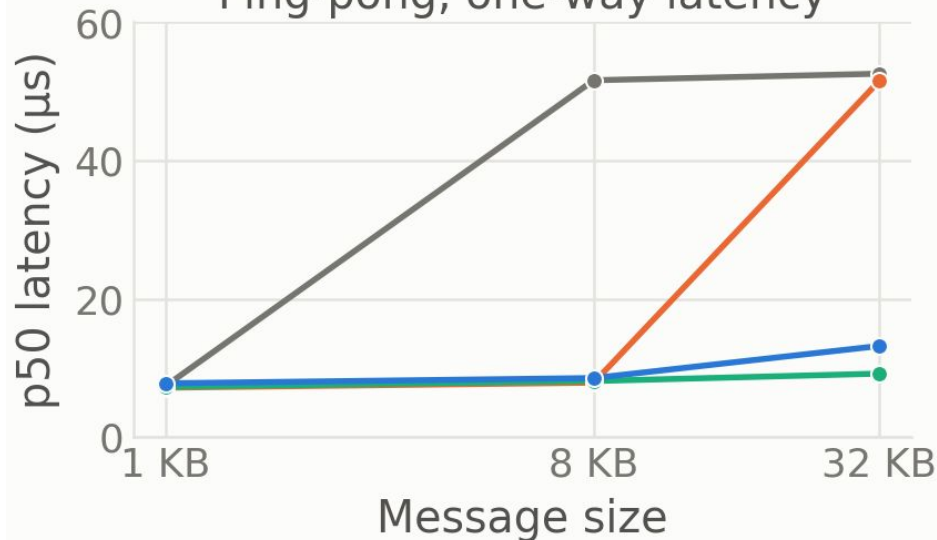
One node: Reconverse matches UCX



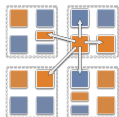
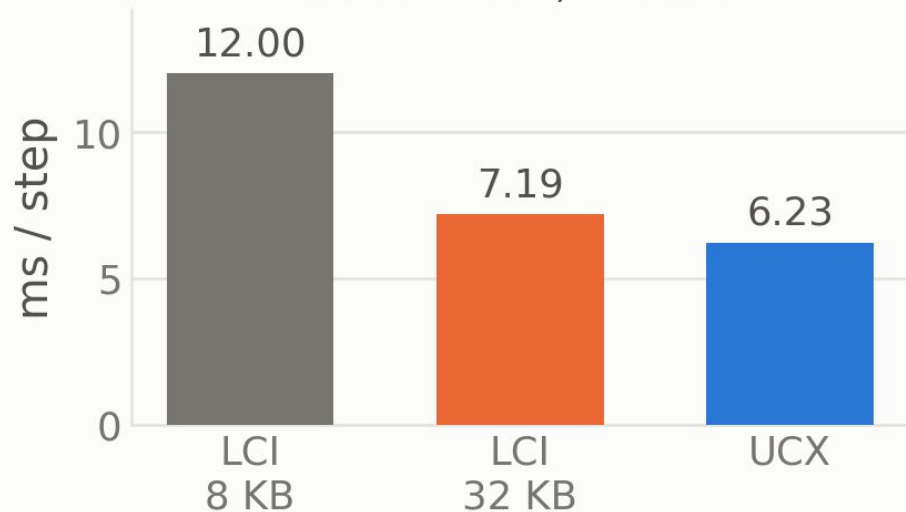
Two nodes: one LCI setting cut 40%

- LCI, 8 KB packets (default)
- LCI, 64 KB packets
- LCI, 32 KB packets
- UCX

Ping-pong, one-way latency

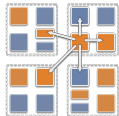
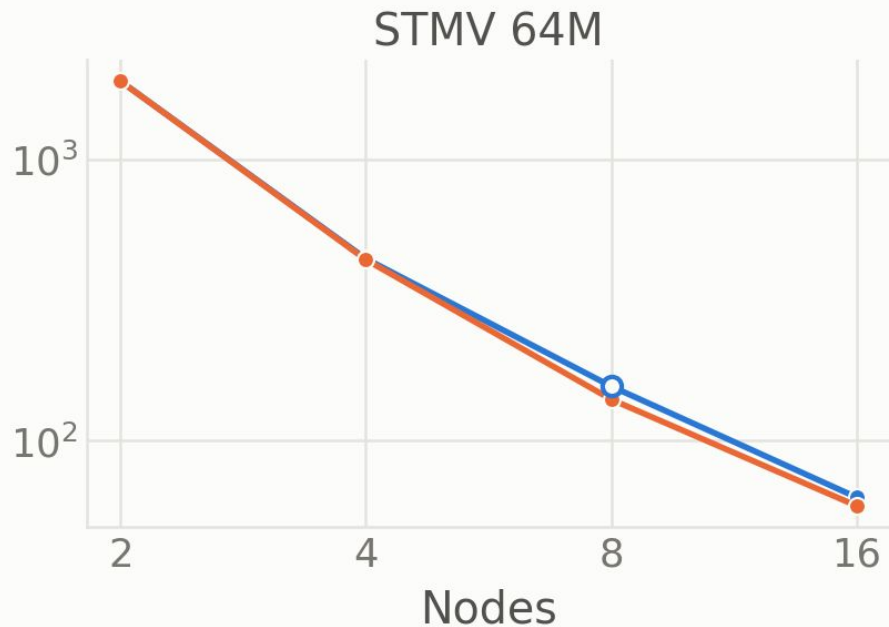
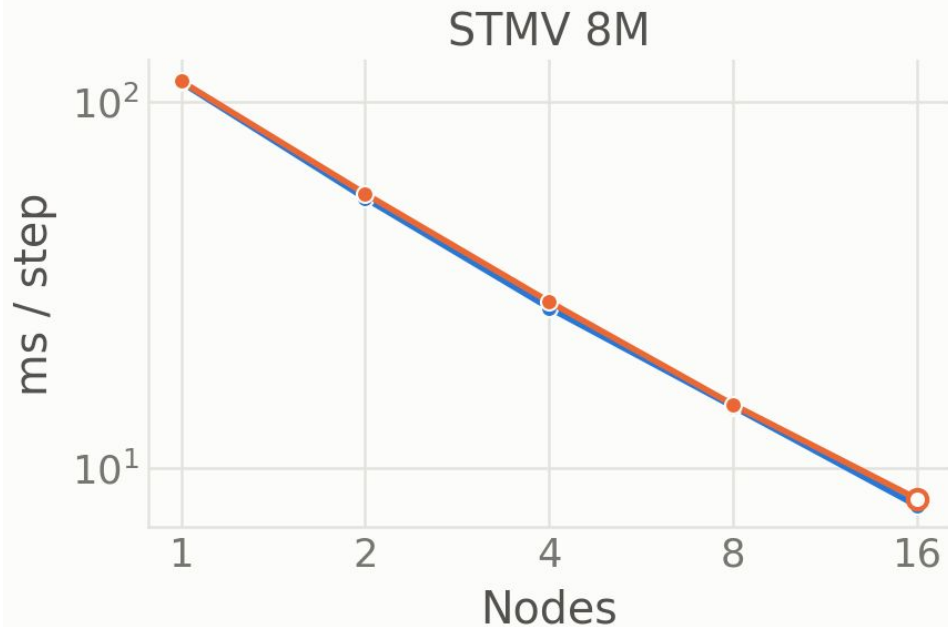


NAMD STMV, 2 nodes



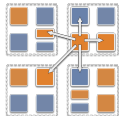
Reconverse tracks UCX to 16 nodes

—● UCX —● Reconverse (LCI) ○ First timing interval only



What Reconverse lets NAMD do next

- Eliminates the communication thread
 - Allows 1 extra core for computation per process
- Enables use of GPU/HAPI API
 - NAMD does not use HAPI, but Reconverse implementation supports D2D RDMA, IPC, and more
 - Also in progress: GPU-aware load balancing
- Simplified communication interface
 - Myriad of comm layers replaced with LCI/LCW
 - Can choose comm layer at runtime
 - No need for separate builds (will need more changes to NAMD build system)



Next steps

- Short term

- Run NAMD at larger scales on Frontier and Aurora
- Performance tuning to see if Reconverse can outright beat existing Converse in performance
- **Charm++ moves to Reconverse as default**
 - We're working on pushing one last release of "classic Charm++" with existing Converse layer
 - After that, all updates are Reconverse only

- Long term

- Rewrite NAMD GPU implementation to rely on Charm++
- Pursue ambitious scaling performance goals (100 us/day?)
- And more...



Questions?

